Formal Language-Based

Universidad Icesi

Computation and Discrete Structures III

Team : Daniel Nastul, Albert Valencia, Nicolas Buelvas

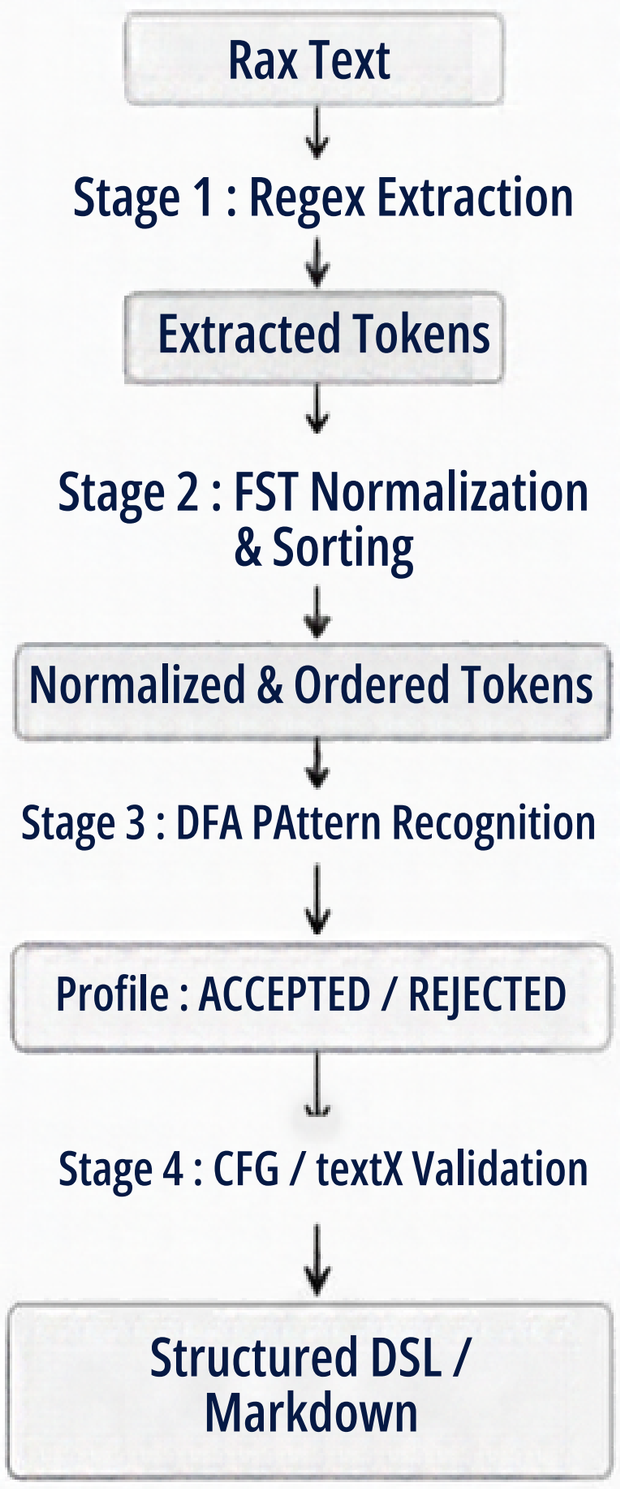
Institution : Universidad Icesi

SECTION 1 : ABSTRACT & PROBLEM STATEMENT

Problem: Recruitment processes require evaluating hundreds of unstructured. it present high variability in syntax, naming conventions, and missing qualifications.

Proposed Solution: ResumeLens, an explainable, deterministic screening architecture leveraging formal language theory.

SECTION 2 : SYSTEM ARCHITECTURE & PIPELINE



SECTION 3 : FORMAL MODELS DEFINITION

Stage 1 : Information Extraction

Identify tokens with Python patterns:
Languages:
JavaScript|JS|Python|Java|TypeScript|C++
ML Frameworks: TensorFlow|PyTorch|Scikit-learn|sklearn|Pandas
Databases:PostgreSQL|Postgres|MySQL|Mongo DB|SQL

Stage 2 : Qualification Normalization (FST)

Formal definition : $M_{FST} = (Q, \Sigma, \delta, \omega, q_{0,} F)$

Normalizes token variations:

- JS → JAVASCRIPT
- sklearn → SCIKIT_LEARN
- postgres → POSTGRESQL

Canonical sorting preps ordering:
Role → Web/Core → Frontend / Data → Backend /Framework → Database → Tools/Git

Stage 3 : Pattern Recognition (DFA)

Formal definition : $M_{DFA} = (Q, \Sigma, \delta, q_0, F)$

Linear state - transition Diagram :

Stage 4: Candidate Profile Language

Defines structured candidate records including personal details, normalized skills, and classification results.
Output generates Markdown summary reports

SECTION 4: EXPERIMENTAL RESULTS & VERIFICATION

94 Automated Test
100%

0%
False Acceptance Rate

Case Study

Python, Pandas, NumPy, Scikit-learn, TensorFlow, SQL, Git → Normalization & DFA → **ACCEPTED** (Machine Learning Engineer)

Case Study (Incomplete)

Missing critical credentials → **REJECTED** (Incomplete Path)

SECTION 5 : CONCLUSION & KEY TAKEAWAYS

Explainability & Rigor: Formal language models eliminate AI hallucination and bias, offering 100% deterministic qualification verification
Modular Architecture: Unified multi-stage pipeline processing heterogeneous input texts into canonical formats and structured DSL specifications.
Robustness: Verified against edge cases through comprehensive E2E test suites with pyformlang and textX.